

CSE2201

Computer Algorithms

Introduction

Proof By Induction

Asymptotic notation

The Course

- Purpose: a rigorous introduction to the design and analysis of algorithms
 - Not a lab or programming course viewpoint
 - Not a math course, either
 - Lab & programming course
- Textbook: *Introduction to Algorithms*, Cormen, Leiserson, Rivest, Stein

The Course

- Grading policy:
 - Class Test [& Assignments]: 20%
 - Participation + Class performance: 8%
 - Final: 72%

The Course

- Format
 - Three lectures/week
 - Assignments
 - Analysis
 - Programming
 - Tests

Why Study Algorithms?

- Important for all other branches of computer science
- Plays a key role in modern technological innovation
- Provides novel “lens” on processes outside of computer science and technology
- Challenging (i.E., Good for the brain!)
- Fun

Cast of characters



Programmer needs to develop a working solution.



Student might play any or all of these roles someday.



Client wants to solve problem efficiently.



Theoretician wants to understand.



Basic **blocking and tackling** is sometimes necessary.
[this lecture]

Proof By Induction

- Claim: $S(n)$ is true for all $n \geq k$
- Basis:
 - Show formula is true when $n = k$
- Inductive hypothesis:
 - Assume formula is true for an arbitrary n
- Step:
 - Show that formula is then true for $n+1$

Induction Example: Gaussian Closed Form

- Prove $1 + 2 + 3 + \dots + n = n(n+1) / 2$

- Basis:

- If $n = 0$, then $0 = 0(0+1) / 2$

- Inductive hypothesis:

- Assume $1 + 2 + 3 + \dots + n = n(n+1) / 2$

- Step (show true for $n+1$):

$$1 + 2 + \dots + n + n+1 = (1 + 2 + \dots + n) + (n+1)$$

$$= n(n+1)/2 + n+1 = [n(n+1) + 2(n+1)]/2$$

$$= (n+1)(n+2)/2 = (n+1)(n+1 + 1) / 2$$

Induction Example: Geometric Closed Form

- Prove $a^0 + a^1 + \dots + a^n = (a^{n+1} - 1)/(a - 1)$ for all $a \neq 1$
 - Basis: show that $a^0 = (a^{0+1} - 1)/(a - 1)$
$$a^0 = 1 = (a^1 - 1)/(a - 1)$$
 - Inductive hypothesis:
 - Assume $a^0 + a^1 + \dots + a^n = (a^{n+1} - 1)/(a - 1)$
 - Step (show true for $n+1$):
$$\begin{aligned} a^0 + a^1 + \dots + a^{n+1} &= a^0 + a^1 + \dots + a^n + a^{n+1} \\ &= (a^{n+1} - 1)/(a - 1) + a^{n+1} = (a^{n+1+1} - 1)/(a - 1) \end{aligned}$$

Induction

- We've been using *weak induction*
- *Strong induction* also holds
 - Basis: show $S(0)$
 - Hypothesis: assume $S(k)$ holds for arbitrary $k \leq n$
 - Step: Show $S(n+1)$ follows
- Another variation:
 - Basis: show $S(0), S(1)$
 - Hypothesis: assume $S(n)$ and $S(n+1)$ are true
 - Step: show $S(n+2)$ follows

Asymptotic Performance

- In this course, we care most about *asymptotic performance*
 - How does the algorithm behave as the problem size gets very large?
 - Running time
 - Memory/storage requirements
 - Bandwidth/power requirements/logic gates/etc.

The challenge

Q. Will my program be able to solve a large practical input?

Why is my program so slow ?

Why does it run out of memory ?



Insight. [Knuth 1970s] Use **scientific method** to understand performance.

Asymptotic Notation

- By now you should have an intuitive feel for asymptotic (big-O) notation:
 - *What does $O(n)$ running time mean? $O(n^2)$? $O(n \lg n)$?*
- Our first task is to define this notation more formally and completely

Analysis of Algorithms

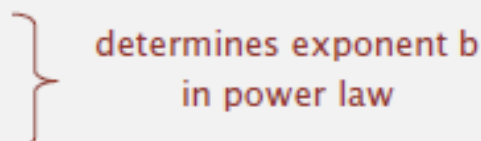
- Analysis is performed with respect to a computational model
- We will usually use a generic uniprocessor random-access machine (RAM)
 - All memory equally expensive to access
 - No concurrent operations
 - All reasonable instructions take unit time
 - Except, of course, function calls
 - Constant word size
 - Unless we are explicitly manipulating bits

Input Size

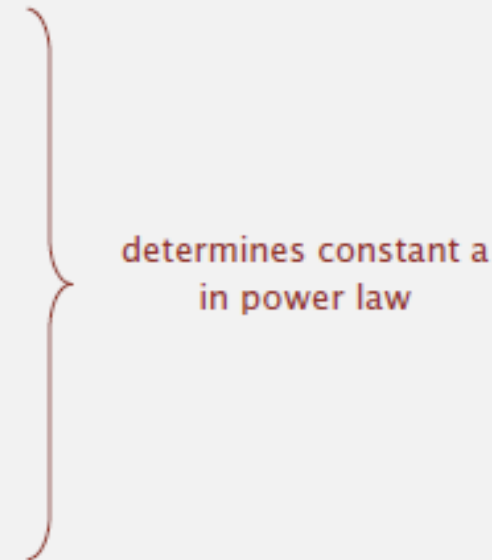
- Time and space complexity
 - This is generally a function of the input size
 - E.g., sorting, multiplication
 - How we characterize input size depends:
 - Sorting: number of input items
 - Multiplication: total number of bits
 - Graph algorithms: number of nodes & edges
 - Etc

Experimental algorithmics

System independent effects.

- Algorithm.
 - Input data.
- 

System dependent effects.

- Hardware: CPU, memory, cache, ...
 - Software: compiler, interpreter, garbage collector, ...
 - System: operating system, network, other apps, ...
- 

determines constant a
in power law

Bad news. Difficult to get precise measurements.

Good news. Much easier and cheaper than other sciences.

operation	example	nanoseconds †
integer add	<code>a + b</code>	2.1
integer multiply	<code>a * b</code>	2.4
integer divide	<code>a / b</code>	5.4
floating-point add	<code>a + b</code>	4.6
floating-point multiply	<code>a * b</code>	4.2
floating-point divide	<code>a / b</code>	13.5
sine	<code>Math.sin(theta)</code>	91.3
arctangent	<code>Math.atan2(y, x)</code>	129.0
...	...	...

† Running OS X on Macbook Pro 2.2GHz with 2GB RAM

operation	example	nanoseconds †
variable declaration	<code>int a</code>	c_1
assignment statement	<code>a = b</code>	c_2
integer compare	<code>a < b</code>	c_3
array element access	<code>a[i]</code>	c_4
array length	<code>a.length</code>	c_5
1D array allocation	<code>new int[N]</code>	$c_6 N$
2D array allocation	<code>new int[N][N]</code>	$c_7 N^2$
string length	<code>s.length()</code>	c_8
substring extraction	<code>s.substring(N/2, N)</code>	c_9
string concatenation	<code>s + t</code>	$c_{10} N$

Running Time

- Number of primitive steps that are executed
 - Except for time of executing a function call most statements roughly require the same amount of time
 - $y = m * x + b$
 - $c = 5 / 9 * (t - 32)$
 - $z = f(x) + g(y)$

Guiding

Won't pay much attention to constant factors,
lower-order terms

Justifications

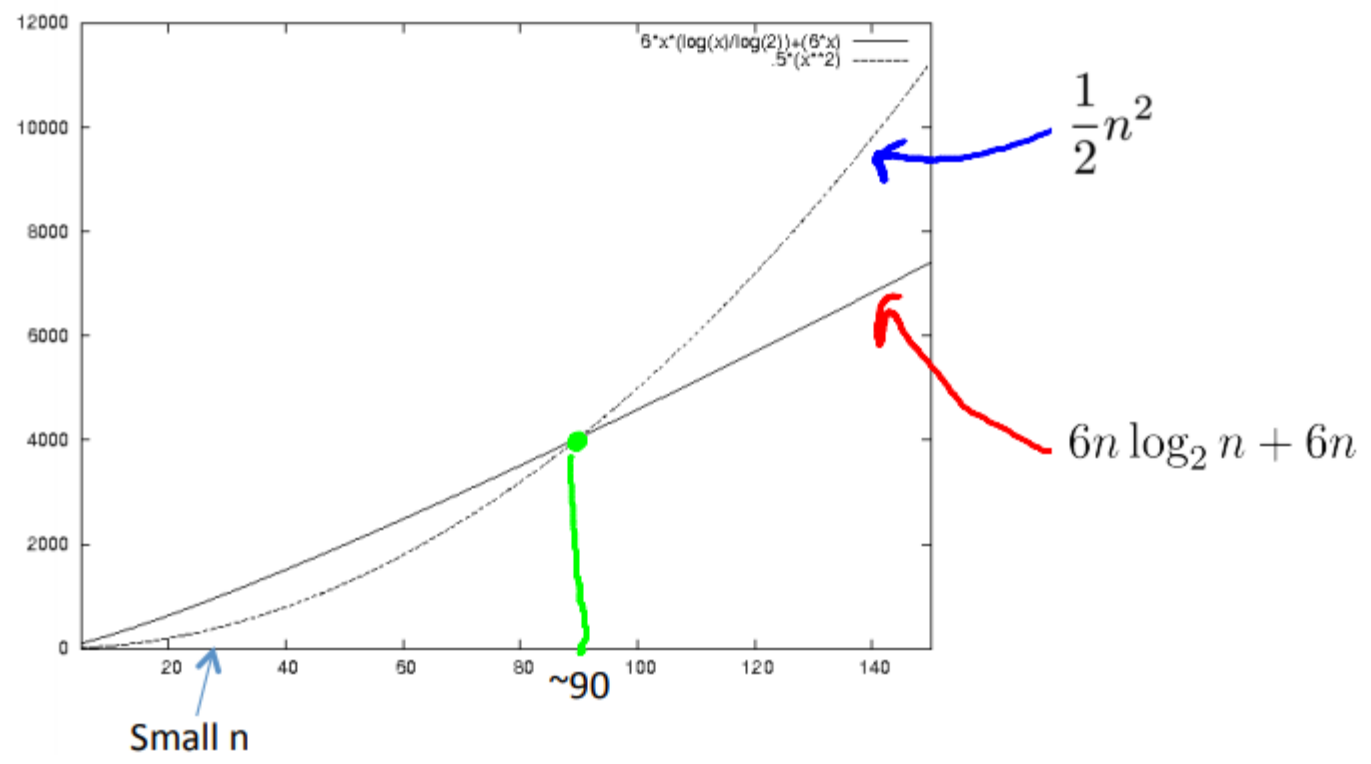
1. Way easier
2. Constants depend on architecture / compiler /
programmer anyways
3. Lose very little predictive power
(as we'll see)

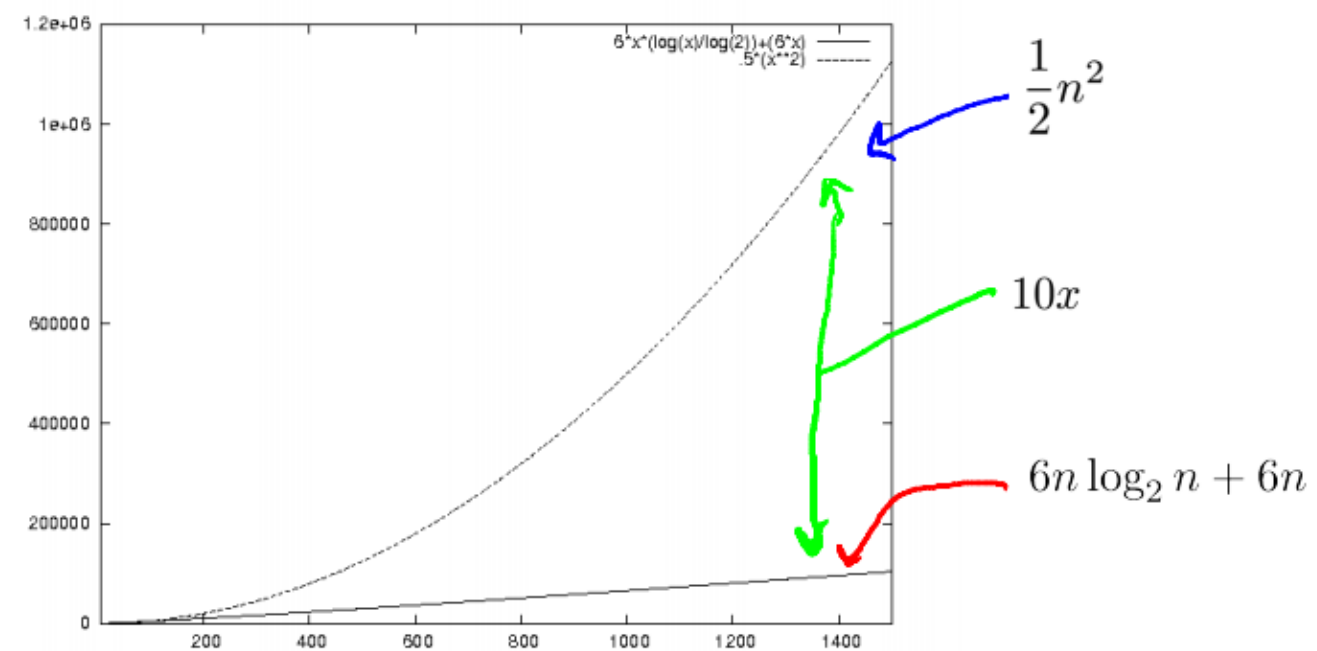
Guiding

Asymptotic Analysis : focus on running time for large input sizes n

Eg : $\underbrace{6n \log_2 + 6n}_{\text{MERGE SORT}}$ “better than” $\underbrace{\frac{1}{2}n^2}_{\text{INSERTION SORT}}$

Justification: Only big problems are interesting!





What Is a “Fast” Algorithm?

This Course : adopt these three biases as guiding principles

fast
algorithm

$\approx$

worst-case running time
grows slowly with input size

Usually : want as close to linear ($O(n)$) as possible

Analysis

- Worst case
 - Provides an upper bound on running time
 - An absolute guarantee
- Average case
 - Provides the expected running time
 - Very useful, but treat with care: what is “average”?
 - Random (equally likely) inputs
 - Real-life inputs

Asymptotic Analysis

High-level idea: Suppress constant factors and lower-order terms

too system-dependent

irrelevant for large inputs

Example: Equate $6n \log_2 n + 6$ with just $n \log n$.

Terminology: Running time is $O(n \log n)$

[“big-Oh” of $n \log n$]

where n = input size (e.g. length of input array).

Example: One Loop

Problem: Does array A contain the integer t ? Given A (array of length n) and t (an integer).

Algorithm 1

```
1: for  $i = 1$  to  $n$  do  
2:   if  $A[i] == t$  then  
3:     Return TRUE  
4: Return FALSE
```

Question: What is the running time? $O(n)$

Example: Two Loops

Given A, B (arrays of length n) and t (an integer). [Does A or B contain t ?]

Algorithm 2

```
1: for  $i = 1$  to  $n$  do  
2:   if  $A[i] == t$  then  
3:     Return TRUE  
4: for  $i = 1$  to  $n$  do  
5:   if  $B[i] == t$  then  
6:     Return TRUE  
7: Return FALSE
```

Question: What is the running time? $O(n)$

Example: Two Nested Loops

Problem: Do arrays A , B have a number in common? **Given** arrays A , B of length n .

Algorithm 3

```
1: for  $i = 1$  to  $n$  do  
2:   for  $j = 1$  to  $n$  do  
3:     if  $A[i] == B[j]$  then  
4:       Return TRUE  
5: Return FALSE
```

Question: What is the running time? $O(n^2)$

Example: Two Nested Loops (II)

Problem: Does array A have duplicate entries? Given arrays A of length n .

Algorithm 4

```
1: for  $i = 1$  to  $n$  do  
2:   for  $j = i+1$  to  $n$  do  
3:     if  $A[i] == A[j]$  then  
4:       Return TRUE  
5: Return FALSE
```

Question: What is the running time? $O(n^2)$

Estimating a discrete sum

Q. How to estimate a discrete sum?

A1. Take discrete mathematics course.

A2. Replace the sum with an integral, and use calculus!

Ex 1. $1 + 2 + \dots + N$.



$$\sum_{i=1}^N i \sim \int_{x=1}^N x \, dx \sim \frac{1}{2} N^2$$

Ex 2. $1^k + 2^k + \dots + N^k$.

$$\sum_{i=1}^N i^k \sim \int_{x=1}^N x^k \, dx \sim \frac{1}{k+1} N^{k+1}$$

The End